PRACTICAL - 10

Aim: Thread-pool runner (TaskRunner.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

TaskRunner.java

```java
public class TaskRunner implements Runnable {
    private final int id;

    public TaskRunner(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        String worker = Thread.currentThread().getName();
        System.out.println("Task " + id + " running on " + worker);
        try {
            Thread.sleep(50);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Driver.java

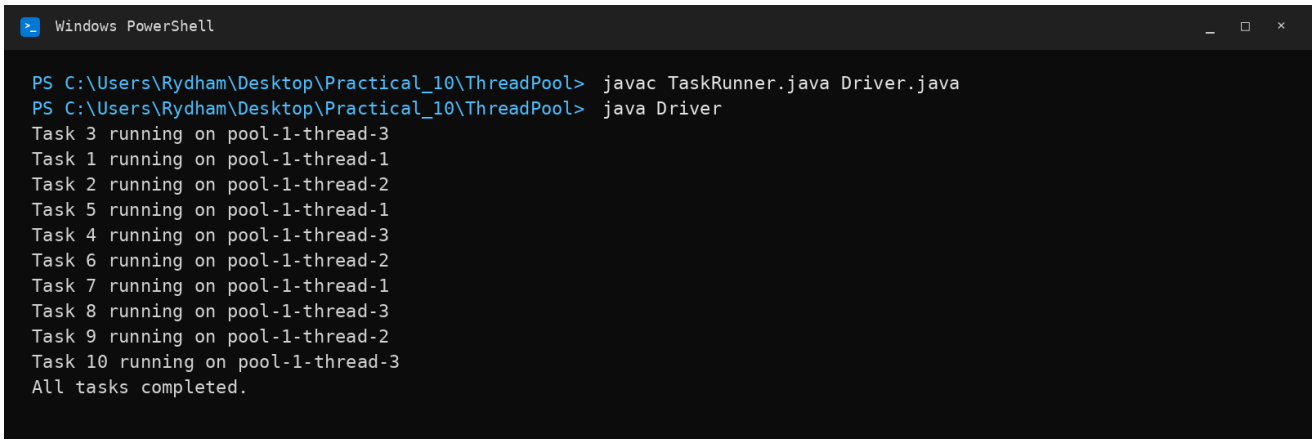
```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Driver {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 10; i++) {
            pool.execute(new TaskRunner(i));
        }

        pool.shutdown();
        if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
            pool.shutdownNow();
        }
        System.out.println("All tasks completed.");
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java concurrency with ExecutorService and fixed thread pools.
- Observed thread recycling across queued asynchronous tasks.
- Managed lifecycle termination using shutdown() and awaitTermination().
- Avoided excessive thread allocation overhead in high-throughput workloads.
- Learned how worker queues decouple task submission from execution.

Aim: Producer-consumer (SharedBuffer.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

SharedBuffer.java

```java
import java.util.ArrayDeque;
import java.util.Queue;

public class SharedBuffer {
    private final Queue<Integer> buffer = new ArrayDeque<>();

    public synchronized void produce(int value) throws InterruptedException {
        while (!buffer.isEmpty()) {
            wait();
        }
        buffer.add(value);
        System.out.println("Produced: " + value);
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (buffer.isEmpty()) {
            wait();
        }
        int value = buffer.remove();
        System.out.println("Consumed: " + value);
        notify();
        return value;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        SharedBuffer buffer = new SharedBuffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.produce(i * 10);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Producer");

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Consumer");

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered inter-thread signaling with wait() and notify().
- Implemented guarded wait loops (while (condition) wait()) to handle spurious wakeups.
- Solved the classic Producer-Consumer synchronization coordination problem.
- Guaranteed zero message loss across asynchronous producer and consumer threads.
- Gained foundational skills for building thread-safe communication pipelines.

Aim: Deadlock demonstration and prevention (DeadlockDemo.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

DeadlockDemo.java

```java
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.CountDownLatch;

public class DeadlockDemo {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    public void showDeadlock() throws InterruptedException {
        CountDownLatch ready = new CountDownLatch(2);
        Thread first = new Thread(() -> lockInOrder(lockA, lockB, ready), "A-to-B");
        Thread second = new Thread(() -> lockInOrder(lockB, lockA, ready), "B-to-A");
        first.setDaemon(true);
        second.setDaemon(true);
        first.start();
        second.start();
        ready.await();

        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] ids = null;
        for (int i = 0; i < 100 && ids == null; i++) {
            ids = bean.findDeadlockedThreads();
            if (ids == null) Thread.sleep(20);
        }
        System.out.println(ids != null
                ? "Deadlock detected: opposite lock order."
                : "Deadlock not detected.");
    }

    private void lockInOrder(Object first, Object second, CountDownLatch ready) {
        synchronized (first) {
            ready.countDown();
            try {
                ready.await();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
            synchronized (second) { }
        }
    }

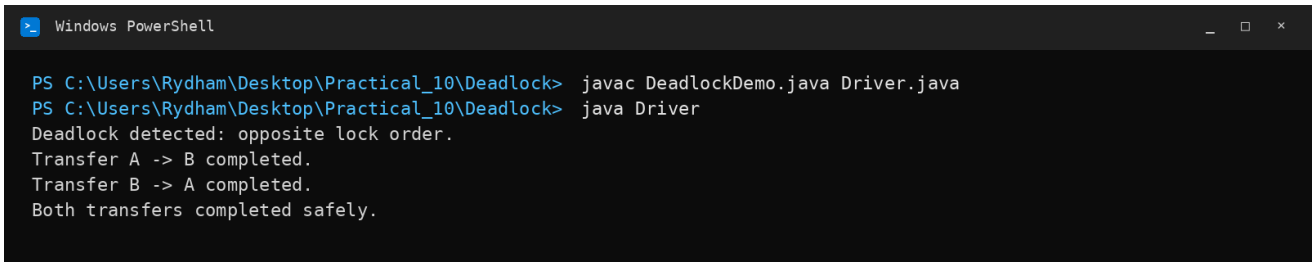
    public void runSafeTransfers() throws InterruptedException {
        Object accountA = new Object();
        Object accountB = new Object();
        Thread first = new Thread(
                () -> transfer(accountA, 1, accountB, 2, "A -> B"));
        Thread second = new Thread(
                () -> transfer(accountB, 2, accountA, 1, "B -> A"));
        first.start();
        second.start();
        first.join();
        second.join();
        System.out.println("Both transfers completed safely.");
    }

    private void transfer(Object from, int fromNo, Object to, int toNo,
                          String label) {
        Object first = fromNo < toNo ? from : to;
        Object second = fromNo < toNo ? to : from;
        synchronized (first) {
            synchronized (second) {
                System.out.println("Transfer " + label + " completed.");
            }
        }
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        DeadlockDemo demo = new DeadlockDemo();
        demo.showDeadlock();
        demo.runSafeTransfers();
    }
}
```

Output Screenshot:



Learning Outcome:

- Reproduced deadlock using opposite object-lock acquisition orders.
- Coordinated competing threads with CountDownLatch.
- Detected deadlocked threads using the ThreadMXBean API.
- Prevented deadlock by acquiring locks in a consistent order.
- Verified that both transfer threads complete safely without hanging.